EPFL - Computer Science

CS-550 - Formal Verification

Lecture 03 — October 27, 2026

Author: Jonathan Dutoit

Professor: Viktor Kuncak

Formal Proofs

Formal Proof System

We will consider some set of logical formulas F (e.g., propositional logic)

Definition. An proof system is (F, *Infer*) where Infer $\subseteq F^* \times F$ a decidable set of *inference* steps.

- a set is *decidable* iff there is a program to check if an element belongs to it
- given a set S, notation S^* denotes all finite sequences with elements from S

We schematically write an inference step $((P_1, ..., P_n), C) \in$ Infer by

$$\frac{P_1 ... P_n}{C}$$

and we say that from $P_1, ..., P_n$ (**premises**) we derive C (**conclusion**).

An inference step is called an *axiom* instance when $n = 0$ (it has no premises).

Definition. Given (F, Infer) where Infer $\subseteq F^* \times F$.

We define a **proof** in (F, Infer) as a finite sequence of inference steps $S_0, ..., S_m \in$ Infer such that, for each S_i where $0 \leq i \leq m$, for each premise P_j of S_i there exists $0 \leq k < i$ such that P_j is the conclusion of S_k.

$$\begin{aligned} S_0 &: ((), && C_0) \\ &... \\ S_k &: ((.....), && \mathbf{P_j}). \\ &... \\ S_i &: ((..., \mathbf{P_j}, ...), && C_i) \end{aligned}$$

Given the definition of the proof, we can replace each premise P_j with the index k where P_j was the conclusion of S_k ($P_j \equiv \text{Conc}(S_k)$)

A proof is then a sequence of elements from $\{0, 1, ...\} * \times F$ where each S_i in the sequence is of form $(k_1, ..., k_n, C)$ for $0 \leq k_1, ..., k_n < i$ and where (Conc(S_{k_1}), ..., Conc(S_{k_n}), C) $\in$ Infer.

This is possible since we can reduce premises to axioms.

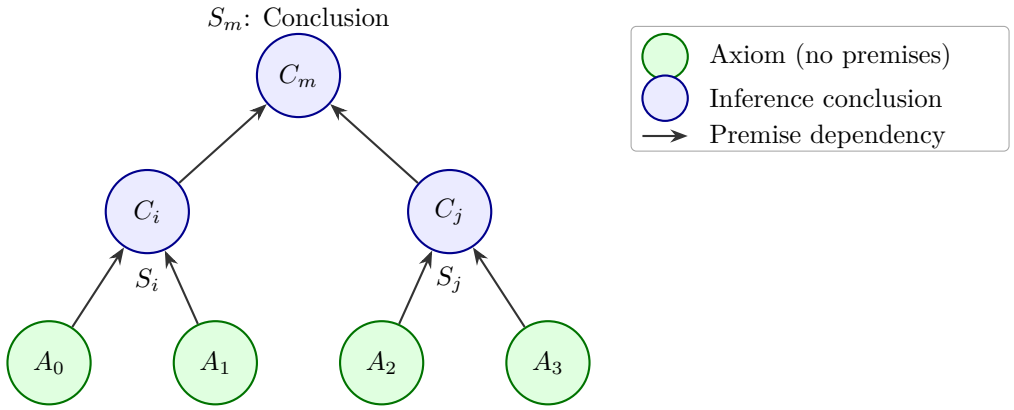


Figure 1: A proof as a directed acyclic graph (DAG). Each node represents the conclusion of an inference step, and each arrow represents a premise dependency.

To check such proof, for each node, follow all its incoming edges backwards in the order of their indices to find the premises, then check that the inference steps in in Infer.

A Minimal Propositional Logic Proof System

Formulas F defined by $F ::= x \,|\, 0 \,|\, F \to F$

Shorthand:
$\neg F \equiv F \to 0$

Inference rules: Infer $= P_2 \cup P_3 \cup MP$ where:

$$\begin{array}{lcll} P_2 & = & \{((), & F \to (G \to F) &) \,|\, F, G \in F\} \\ P_3 & = & \{((), & ((F \to (G \to H)) \to ((F \to G) \to (F \to H)) &) \,|\, F, G, H \in F\} \\ MP & = & \{((F \to G, F), & G &) \,|\, F, G \in F\} \end{array} .$$

Elements of P_1, P_2, P_3 are all axioms. These are infinite sets, but are given a schematic way and there is an algorithm to check if a given formula satisfies each of the schemas.

Exercise: proof of $a \to a$

Proof system:

$$P_2 : F \to (G \to F) \qquad P_3 : ((F \to (G \to H)) \to ((F \to G) \to (F \to H)) \qquad MP : \frac{F \to G, F}{G}$$

We start by assuming F $\leftrightarrow a$, $G \leftrightarrow (a \to a)$, $H \leftrightarrow a$

1.	$(a \to ((a \to a) \to a)) \to ((a \to (a \to a)) \to (a \to a))$	Instance of P_3
2.	$(a \to ((a \to a) \to a)$	Instance of P_2
3.	$(a \to (a \to a)) \to (a \to a)$	MP on 1 and 2
4.	$a \to (a \to a)$	Instance of P_2
5.	$a \to a$	MP on 3 and 4

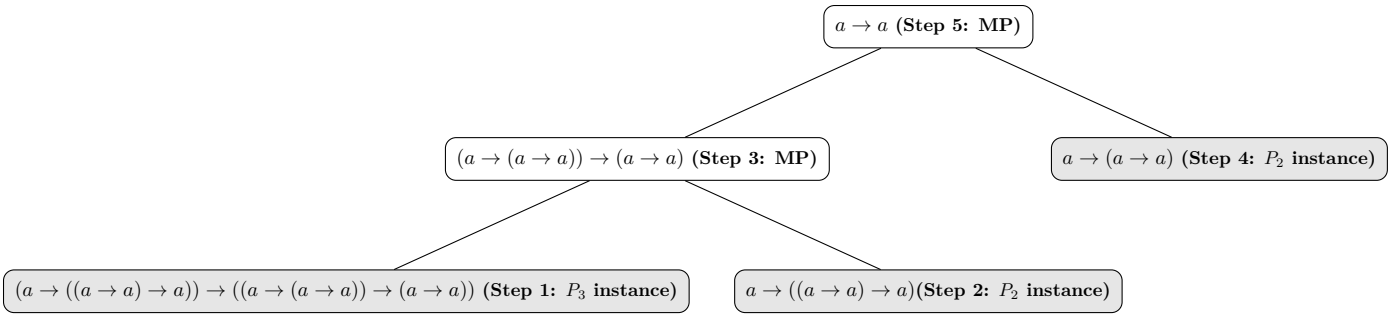


Figure 2: DAG proof structure for $a \to a$ using Modus Ponens (MP) and axiom instances P_2 and P_3.

Soundness of a Proof System

Definition. Given (F, Infer), $\text{Infer} \subseteq F^* \times F$ **and a set of assumptions** $A \subseteq F$, a **derivation from** A in (F, Infer) is a proof in $(F, \text{infer'})$ where:

$$\text{Infer}' = \text{Infer} \cup \{((), F) \mid F \in A\}$$

Thus, assumptions from A are treated just as axioms. *Hence, A expands the proof system by treating every formula in A as an extra axiom (i.e, a premise-free rule that can be introduced at any point in the proof).*

Definition. We say that $F \in F$ is provable from assumptions A, denote $\vdash_{\text{Infer}} F$ iff there exists a derivation from A in Infer that contains an inference step whose conclusion is F.

We write $\vdash_{\text{Infer}} F$ to denote that there exists a proof in Infer containing F as a conclusion (same as $\varnothing \vdash_{\text{Infer}} F$).

This is syntactic process. It relies purely on symbol manipulation and matching rules, without caring about what the symbols actually mean. It means that the formula F can be mechanically derived using the proof system's inference rules.

Definition. Given a set $A \subseteq F$ where F are in propositional logic, and $C \in F$, we say that C is a **semantic consequence** of A, denote $A \models C$ iff for every environment e that defines all variables in $FV(C) \cup \bigcup_{P \in A} FV(P)$ if $[\![P]\!]_e = 1$ for all $P \in A$, then $[\![C]\!]_e = 1$.

$A \models C$ means the conclusion is guaranteed to be true whenever the premises are true. In other words, there is not counterexample where all premises are true and the conclusion is false.

Definition. Given (F, Infer) where F are propositional, step $((P_1...P_n), C) \in$ Infer is **sound** iff $\{P_1, ..., P_n\} \models C$. Proof system Infer is sound if every inference step is sound.

For axioms, this definition reduces to saying that C is true for all interpretations, i.e, that C is a valid formula (tautology).

Definition. Let (F, Infer) where F are propositional logic formulas. If every inference rule in Infer is sound, then $A \vdash_{\text{Infer}} F$ implies $A \models F$.

Proof is immediate by induction on the length of the formal proof.
Consequence: $\vdash_{\text{Infer}} F$ implies F is a tautology.

Semantic vs Syntactic Consequence

Semantic consequence: We check whether $\{A_1\} \models G$ by checking all possible interpretations: in every propositional assignment (any "world" we can imagine) if A_1 is true, then so if G. For propositional logic, a world gives truth values (0 or 1) for the variables.

Syntactic consequence (provability): We check whether $\{A_1\} \vdash G$ by finding a derivation of G with A_1 as an additional axiom. Syntactic consequence is specific for a particular proof system.

In general we have a set of assumptions, $A = \{A_1, A_2, ...\}$

For a given proof system, we have two possible conclusions:

- soundness: if $A \vdash G$, then $A \models G$
  (without this, our proof system is "broken")
- completeness: if $A \models G$, then $A \vdash G$
  (without this, there are true things we cannot prove, which may be OK)

Completeness

Definition. A proof system is **complete** if it derives all formulas we want it to derive. Completeness for classical propositional logic means that the system derives all propositional tautologies.*Meaning every tautology can be proven using these rules.*

Example: Is our example system complete with respect to formulas built from *to* and 0?

$$P_2 : F \to (G \to F) \qquad P_3 : ((F \to (G \to H)) \to ((F \to G) \to (F \to H)) \qquad MP : \frac{F \to G, F}{G}$$

- In particular, if we are given an arbitrary valid propositional formula (propositional tautology), is there always a way to derive it using the axioms and the MP rule?
- Does there exist a tautology that is not reachable from axioms using MP rule?
- Is valid formula $0 \to a$ provable?

The axioms don't say anything about the 0 constant. It is as if it was any other subformula. Take proof:

$$\begin{array}{l} \ldots \\ a \to a \\ (a \to a) \to (0 \to (a \to a)) \quad \text{(Axiom)} \\ 0 \to (a \to a) \quad \text{(MP)} \end{array}.$$

Replace 0 with e.g., $b \to c$ in the proof above. It will get a valid proof:

$$\begin{array}{l} \ldots \\ a \to a \\ (a \to a) \to ((b \to c) \to (a \to a)) \quad \text{(Axiom)} \\ (b \to c) \to (a \to a) \quad \text{(MP)} \end{array}.$$

Replacing 0 with some fixed formula gives us valid proofs.

Now let's suppose that our proof system can prove $0 \to a$. Then it can also prove a formula resulting by 0 with $a \to a$, so there would exists a proof of $(a \to a) \to a$. We have shown before that we can derive $a \to a$, so we would also derive a using MP. But a is not a tautology, which is a contradiction with the fact that the system is sound.

Note: The Sheffer Stroke which represents the NAND operator has properties:

- **Functional completeness**: can express any truth table on its own, i.e, it build all other logical connectives (like AND, OR, NOT, IMPLIES).
- **Deductive completeness**: Once properly formulated into an axiomatic system, a Sheffer stroke proof system is deductively complete for classical propositional logic. Every tautology can be successfully derived using its rules.

Towards Propositional Resolution

Consider propositional formulas with $\wedge, \vee, \neg$

Case analysis proof rule. $\{((F, G), F[x := 0] \vee G[x := 1]) \mid F, G \in F, x - \text{variable}\}$:

$$\frac{F \qquad G}{F[x := 0] \vee G[x := 1]}.$$

Proof of soundness. To show $\{F, G\} \models (F[x := 0] \vee G[x := 1])$, consider an environment e and assume $[\![F]\!]_e = 1$ and $[\![G]\!]_e = 1$.

- If *e(x) = 0*, then $[\![F[x := 0]]\!]_e = [\![F]\!]_e = 1$, so the first disjunct is 1.
- If *e(x) = 1*, then $[\![G[x := 1]]\!]_e = [\![G]\!]_e = 1$, so the second disjunct is 1.

In both cases, the disjunction evaluates to 1 in e.

Simplification rules that preserve equivalence: $0 \wedge F \rightsquigarrow 0$, $F \wedge 0 \rightsquigarrow 0$, $1 \wedge F \rightsquigarrow F$, $F \wedge 1 \rightsquigarrow F$, $0 \vee F \rightsquigarrow F$, $F \vee 0 \rightsquigarrow F$, $1 \vee F \rightsquigarrow 1$, $F \vee 1 \rightsquigarrow 1$, $\neg 0 \rightsquigarrow 1$, $\neg 1 \rightsquigarrow 0$.

Introduce inferences $\{((F), F') \mid F' \text{ is simplified } F\}$. These rules are also sound. Let Infer_D be the proof system with the above case analysis and simplification rules.

Example Derivation from $A = \{a \wedge b, \neg b \vee \neg a\}$. We want to show $A \vdash 0$. This is called proof by refutation. In automated reasoning, formal verification, and SAT solvers, showing that a set of assumptions leads to 0 is one of the most powerful techniques available, known as refutation or **proof by contradiction**.

Actually, the assumptions in A are completely unsatisfiable. There is no possible truth assignment (world) where all formulas in A can be true at the same time.

A naturally leads to a contradiction:

- Assumption 1 $a \wedge b$: Tells you that both a is true and b is true.
- Assumption 2 $\neg b \vee \neg a \equiv \neg(a \wedge b)$ (De Morgan's laws) tells that it is not the case that both are true.

Using inference rules (*case analysis* or *resolution rules*, the system clashes these opposing facts until everything cancels out, leaving you with 0.

The **ultimate "success" state** of a resolution algorithm is deriving the empty clause (0). Deriving 0 acts as a certificate proving that your initial set of constraints or code specifications contains a bug, a dead end, or a logical contradiction.

> **Definition.** A *set* A of formulas is *satisfiable* if there exists e such that, for every $F \in A, [\![F]\!]_e = 1$
>
> When $A = \{F_1, ..., F_n\}$ (finite), the notion is the same as the satisfiability of $F_1 \wedge ... \wedge F_n$.
>
> Otherwise, we call the set A *unsatisfiable*.

Theorem (Soundness Consequence)

> *If* $A \vdash_{\text{Infer}_D} 0$ *then* A *is unsatisfiable.*
>
> If there exists e such that *e(F) = 1* for all $F \in A$ then by soundness of Infer_D, *e(0) = 1*, a contradiction. So there is no such e.

Theorem (Refutation Completeness)

> *If a finite set* A *is unsatisfiable*, then $A \vdash_{\text{Infer}_D} 0$
>
> Proof hint: take conjunction of formulas in A and existentially quantify it to get *A'*. What is the relationship of the truth of *A'* and the satisfiability of A? For a conjunction for formulas F, can you express $.F$ using Infer_D?

Illustration of completeness: Let $A = \{F_1, F_2\}$ and let $FV(F_1) \cup FV(F_2) = \{x_1, ..., x_n\}$ and let x be some x_i. We have the following equivalences:

$$\begin{aligned}
& \exists x.(F_1 \wedge F_2) \\
\equiv\ & (F_1 \wedge F_2)[x := 0] \vee (F_1 \wedge F_2)[x := 1] \\
\equiv\ & (F_1[x := 0] \wedge F_2[x := 0]) \vee (F_1[x := 1] \wedge F_2[x := 1]) \quad . \\
\equiv\ & (F_1[x := 0] \vee F_1[x := 1]) \wedge (F_1[x := 0] \vee F_2[x := 1]) \wedge \\
& (F_2[x := 0] \vee F_1[x := 1]) \wedge (F_2[x := 0] \vee F_2[x := 1])
\end{aligned}$$

Where we try both values meaning of substitution. This is like doing the case analysis.

Existentially quantifying over a variable gives us result of applying decision rule to all pairs of formulas F_1, F_2.

Systematically applying rules will derive formula Z equivalent to $\exists x_1 .... \exists x_n . (F_1 \wedge F_2)$.

When A is unsatisfiable, Z is equivalent to 0, and has no variables. By simplification rules, we can derive 0.

Resolution on Clauses

We previously considered a "simple" proof system that's easy to understand. In practice, we often don't work with arbitrary formulas, but with clauses since they can be represented more efficiently.

Definitions

- A propositional ***literal*** is either a variable (e.g., s) or its negation ($\neg x$).
- A ***clause*** is a disjunction of literals.
  For convenience, we can represent clause as a finite *set of literals* (because of associativity, commutativity, and idempotence of $\vee$).

If C is a clause then $[\![C]\!]_e = 1$ iff there exists a literal $L \in C$ such that $[\![L]\!]_e = 1$.

We represent 0 using the empty clause $\varnothing$ (represents **contradiction**)

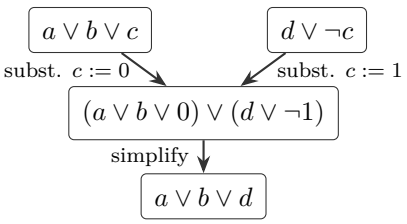
As for any formulas, a finite set of clauses A can be interpreted as a conjunction. Thus, a set of clauses can be viewed as a formula in conjunctive normal form:

$$A = \{\{a\}, \{b\}, \{\neg a, \neg b\}\}$$

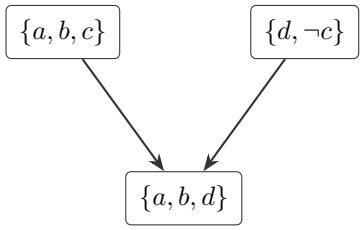
represents the formula

$$a \wedge b \wedge (\neg a \vee \neg b)$$

Case analysis



Resolution analysis



Clausal resolution rule (decision rule for clauses):

$$\frac{C_1 \cup \{x\} \qquad C_2 \cup \{\neg x\}}{C_1 \cup C_2}$$

Theorem (Soundness)

> *Clausal resolution is sound: for all clauses C_1, C_2 and propositional variable x,* $\{C_1 \cup \{x\}, C_2 \cup \{\neg x\}\} \models C_1 \cup C_2$

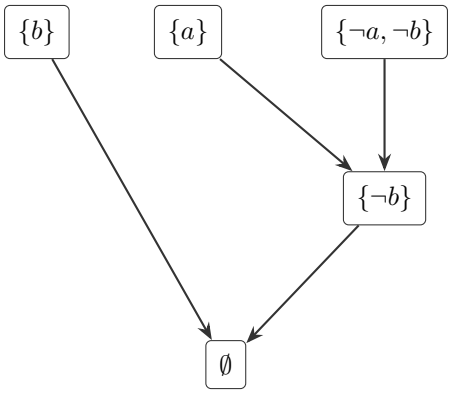
Theorem (Refutational Completeness)

> *A finite set of clauses A is unsatisfiable if and only if there exists a derivation of the empty clause from A using clausal resolution.*

Example: Use resolution to prove that the following formula is valid:

$$\neg(a \wedge b \wedge (\neg a \vee \neg b))$$

Prove that its negation is unsatisfiable set of clauses:



Unit Resolution

> **Definition.** A *unit clause* is a clause that has precisely one literal; it's of the form $\{L\}$
> Given a literal L, its complement $\bar{L}$ is defined by $\bar{x} = \neg x, \quad \overline{\neg x} = x$.
>
> **Unit resolution** is a special case of resolution where at least one of the clauses is a unit clause:
>
> $$\frac{C \qquad \{L\}}{C \setminus \{\bar{L}\}}$$
>
> Soundness: if L is true, then $\bar{L}$ is false, so it can be deleted from a disjunction C.

Subsumption: when applying resolution, if we obtain a clause $C' \subseteq C$ that is subset of a previously derived one, we can delete C so we do not consider it any more. Any use of C can be replaced by use of C' with progress towards $\emptyset$ at least as good.

Conclusion: if we derive $\{L\}$ we can remove all other occurrences of L and $\bar{L}$: if $L \in C$ then C is subsumed by $\{L\}$ and if $\bar{L} \in C$ then C is subsumed by $C \setminus \{\bar{L}\}$.

From Formulas to Clauses

Normal Form Transformation

Transformation of an arbitrary formula into a set of clauses is exponential in general, applying from left to right equivalence

$$F_1 \wedge (F_2 \wedge F_3) \leftrightarrow (F_1 \vee F_2) \wedge (F_1 \vee F_3)$$

duplicates sub-formulas F_1, which may result in an exponential larger formula if for instance F_1 is not a propositional variable.

Key idea: Transform formula into *disjunctive* normal form, because checking satisfiability of formula in disjunctive normal form is trivial, such formula is a disjunction of conjunctions D_i and we have these equivalences:

$$\begin{gathered} \exists e. [\![D_1 \vee ... \vee D_n]\!]_e = 1 \\ \exists e. ([\![D_1]\!]_e == 1 \vee ... \vee [\![D_n]\!]_e = 1) \\ (\exists e. [\![D_1]\!]_e = 1) \vee ... \vee (\exists e. [\![D_n]\!]_e = 1) \end{gathered} .$$

and the last condition is trivial to check, because we check satisfiability of conjunction D_i separately.

> **Definition.** Formula F_1 and F_2 are **equivalent** iff: $F_1 \models F_2$ and $F_2 \models F_1$ $(\forall e. [\![F_1]\!]_e = [\![F_2]\!]_e)$

> **Definition.** Formulas F_1 and F_2 are **equisatisfiable** iff: F_1 is satisfiable whenever F_2 is satisfiable.

Note: Equivalent formulas are always equisatisfiable, but the converse is not the case in general.

Example: consider these two formulas:

$$F_1 : (a \wedge b) \vee c \qquad F_2 : (x \leftrightarrow (a \wedge b)) \wedge (x \vee c)$$

They are equisatisfiable but not equivalent. For example, given $e = \{(a,1),(b,1),(c,0),(x,0)\}$, $[\![F_1]\!]_e = 1$ whereas $[\![F_2]\!]_e = 0$. Interestingly, every choice of a, b, c that makes F_1 true can be extended to make F_2 true appropriately, if we chose x as $[\![a \wedge b]\!]_e$.

Flatenning as Satisfiability Preserving Transformation

Observation: Let F be a formula, G another formula, and $x \notin FV(F)$ a propositional variable. Let $F[G := x]$ denote the result of replacing an occurrence of formula G inside F with x. Then F is equisatisfiable with

$$(x = G) \wedge F[G := x]$$

where $=$ denotes $\leftrightarrow$.

Strategy: apply transformation from smallest non-variable subformulas. **Tseytin's Transformation**!

Consider formula with $\neg, \wedge, \vee, \rightarrow, =, \oplus$

- Replace $F_1 \rightarrow F_2$ with $\neg F_1 \vee F_2$ and push negation into the propositional variables using De Morgan's laws and switching between $\oplus$ and $=$.
- Repeat: flatten an occurrence of a binary connective whose arguments are literals

- In the resulting conjunction, express each equivalence as a conjunction of clauses:

conjunct	corresponding clauses
$x = (a \wedge b)$	$\Leftrightarrow (x \to (a \wedge b)) \wedge ((a \wedge b) \to x)$ $\Leftrightarrow (\neg x \vee (a \wedge b)) \wedge (\neg(a \wedge b) \vee x)$ $\Leftrightarrow (\neg x \vee a) \wedge (\neg x \vee b) \wedge (\neg a \vee \neg b \vee x)$ $\{\bar{x}, a\}, \{\bar{x}, b\}, \{\bar{a}, \bar{b}, x\}$
$x = (a \vee b)$	$\{\bar{x}, a, b\}, \{\bar{a}, x\}, \{\bar{b}, x\}$
$x = (a = b)$	$\Leftrightarrow (x \to (a = b)) \wedge ((a = b) \to x)$ $\Leftrightarrow (\neg x \vee ((a \to b) \wedge (b \to a)) \wedge (\neg((a \to b) \wedge (b \to a)) \vee x)$ $\Leftrightarrow (\neg x \vee ((\neg a \vee b) \wedge (a \vee \neg b)) \wedge (\neg((\neg a \vee b) \wedge (a \vee \neg b)) \vee x)$ $\{\bar{x}, \bar{a}, b\}, \{\bar{x}, a, \bar{b}\}, \{x, a, b\}, \{x, \bar{a}, \bar{b}\}$

Example: Find an Equisatisfiable Set of Formulas in CNF

$$\{\boxed{c \wedge a} \vee (\neg c \wedge b)\}$$

$$\{x_1 \vee \boxed{\neg c \wedge b},\ x_1 \leftrightarrow (c \wedge a)\}$$

$$\{x_1 \vee x_2,\ x_2 \leftrightarrow (\neg c \wedge b),\ x_1 \leftrightarrow (c \wedge a)\}$$

$$\{x_1 \vee x_2,\ x_2 \to (\neg c \wedge b),\ (\neg c \wedge b) \to x_2,\ x_1 \to (c \wedge a),\ (c \wedge a) \to x_1\}$$

$$\{x_1 \vee x_2,\ \neg x_2 \vee \neg c,\ \neg x_2 \vee b,\ c \vee \neg b \vee x_2,\ \neg x_1 \vee c,\ \neg x_1 \vee a,\ \neg c \vee \neg a \vee x_1\}$$

When representing clauses as sets:

$$\{\{x_1, x_2\},\ \{\neg x_2, \neg c\},\ \{\neg x_2, b\},\ \{c, \neg b, x_2\},\ \{\neg x_1, c\},\ \{\neg x_1, a\},\ \{\neg c, \neg a, x_1\}\}$$

Finding Proofs

SAT Solvers

A SAT solver takes as input a set of clauses.

- To check **satisfiability**, convert to equisatisfiable set of clauses in polynomial time using Tseytin's transformation.
- To check **validity** of formula, take negation, check satisfiability, then negate the answer.

How should we check satisfiability of a set of clauses?

- Resolution on clauses, favoring unit resolution and applying subsumption (complete) - *Davis and Putnam, 1960.*
- Truth table method: check one value of a variable, then other (space efficient).